Learn Python for Automation

Lesson 6: Working with the OS and Paths

Working with the OS and Paths - Study Notes

Key Concepts

1. **`os` vs `pathlib`** – Two complementary APIs for file system manipulation.
2. **Path objects** – `pathlib.Path` provides an object oriented interface to file paths.
3. **Cross platform compatibility** – Using `os.path` or `pathlib` abstracts OS specific separators and conventions.
4. **File system operations** – Listing, moving, renaming, creating, and deleting files/directories.
5. **Error handling** – Common pitfalls (e.g., `FileNotFoundError`, `PermissionError`) and how to catch them.

Summary

Python's standard library gives you two powerful ways to interact with the operating system's file system: the procedural `os` module and the modern, object oriented `pathlib` module. `os` offers functions like `listdir`, `rename`, and `mkdir`, while `pathlib` wraps these operations into methods on `Path` objects, making code more readable and less error prone.

Both modules let you list directory contents, move or rename files, and create nested directories. `pathlib` automatically handles path separators, joins, and even provides convenient methods such as `mkdir(parents=True, exist_ok=True)` to create complex directory trees in a single call. Understanding when to use each approach—and how they interoperate—is essential for writing robust, cross platform scripts.

Important Points

• **Importing**

```python
import os
from pathlib import Path
```

• **Listing files**

- `os.listdir(path)` returns names only.
- `os.scandir(path)` yields `DirEntry` objects with stat info.
- `Path.iterdir()` yields `Path` objects for each entry.

• **Moving/Renaming**

- `os.rename(src, dst)` or `shutil.move(src, dst)` for cross drive moves.
- `Path.rename(new_path)` or `Path.replace(new_path)` for atomic replacement.

• **Creating directories**

- `os.makedirs(path, exist_ok=True)` – creates intermediate dirs.
- `Path.mkdir(parents=True, exist_ok=True)` – same, but with a `Path` object.
    • **Path manipulation**
- `Path / 'subdir' / 'file.txt'` for joining.
- `.parent`, `.stem`, `.suffix`, `.parts` for dissecting paths.
    • **Cross platform safety**
- Avoid hard coding separators (`/` or `\`); use `Path` or `os.path.join`.
- Use `os.sep` or `Path`'s `.as_posix()`/`.as_uri()` when needed.
    • **Error handling**

```python
try:
    os.rename('old.txt', 'new.txt')
except FileNotFoundError:
    print("Source file missing")
```

    • **Cleaning up**
- `shutil.rmtree(path)` to delete non empty directories.
- `Path.unlink()` to delete a single file.

Examples

1. Listing all Python files in a directory

```python
from pathlib import Path
folder = Path('projects')
py_files = [p for p in folder.iterdir() if p.suffix == '.py']
for py in py_files:
    print(py.name)
```

Explanation:

`Path.iterdir()` yields `Path` objects for each entry. The list comprehension filters only those whose suffix is `.py`. Printing `py.name` shows just the filename.

2. Moving a file into a new subdirectory, creating it if needed

```python
import shutil
from pathlib import Path
src = Path('data/raw.csv')
dest_dir = src.parent / 'processed'
dest_dir.mkdir(parents=True, exist_ok=True)   # create processed/
new_name = dest_dir / 'raw_processed.csv'
shutil.move(str(src), str(new_name))
```

```
print(f"Moved to {new_name}")
```

Explanation:

- `mkdir(parents=True, exist_ok=True)` ensures the `processed` folder exists.
- `shutil.move` handles cross drive moves; we convert `Path` objects to strings because `shutil.move` accepts path-like objects but works reliably with strings.
- The new file is renamed to `raw_processed.csv` in the destination directory.

Quick Review

1. What is the difference between `os.listdir()` and `Path.iterdir()`?
2. How do you create nested directories with a single command in `pathlib`?
3. Which function would you use to move a file across different drives?
4. Explain why `Path / 'sub' / 'file.txt'` is preferred over string concatenation.
5. What does `exist_ok=True` do when creating a directory?

Further Reading

- `shutil` module – advanced file operations (copy, archive, disk usage).
- `os.path` utilities – `join`, `split`, `normpath`, `abspath`.
- `pathlib` advanced features – `Path.glob`, `Path.rglob`, `Path.read_text()`.
- File permissions and ownership with `os.chmod` / `stat` module.
- Working with temporary files/directories (`tempfile` module).